Memory in the Twelve-Engine Hierarchy

Oct 16, 2026 · @Hermes Toph

Summary

The proposal works as a memory architecture. Below are the three results and the caveat that limits each one.

- **Implied fourth = parity engine (M1).** Each group of three data engines plus its implied fourth satisfies $H_1H_2H_3H_4 = 1$, so any one of the four can be rebuilt exactly from the other three. The hierarchy survives the loss of one engine per group (three in all), but it costs 25% of capacity: only 9 of the 12 engines carry new data. Two losses in the same group cannot be recovered.
- **Offload onto the inverse tier system is exact (M2).** The tier transition word U, U^{-1}, U, U^{-1} is its own reversed inverse, so the exact inverse tier system is the same tier system run backwards. The net rotation is the identity, the spinor product is exactly 1, and the radial scale goes 31/3 → 3/31. Rotating Hurwitz-integer data through a tier keeps its norm and is undone exactly, so nothing grows in bit width. **Caveat:** offloading an implied fourth gives back only the product of its three parents, not their separate histories.
- **The free 120° phase (M3).** If the three phase angles are three separate time slots, the third slot is a genuinely free channel: +50% over two. If they are three phasors on one complex carrier, it is not free ($1 + \omega + \omega^2 = 0$). Under the 120° rotation, three slots split into one invariant line plus one rotating plane. The invariant line is exactly the position of the implied fourth (the coset the rotation fixes). **Recommended use:** carry the implied-fourth (parity) traffic, or the reference clock, in the third phase so both data phases stay 100% data.
- **Counts.** 12 × 6 = 72 tier slots and 12 × 60 = 720 outer frames match the 72 middle-gear and 720 outer frames. These are counting correspondences, not theorems.
- **Limits that remain.** Pigeonhole bounds still apply: the hierarchy adds erasure tolerance and phase throughput, not stored information beyond the bits actually written.

Every claim below is checked in `verify_memory.py` (log in §6).

1. Reading of the proposal

The analysis rests on these assumptions. Each one is the most direct reading of the request in terms of the fifteen lemmas.

- **A1 — six tiers.** "6-tier" means the core plus the five toroidal tiers of Lemma 1 ($R_n = 5n^2+15n+15$, $W_n = 16-2n$, $W\Sigma = 60$). The "5-manifold" label does not enter any statement below. If it is meant to add structure, that is listed as open item O3.
- **A2 — engine state.** An engine's memory state is its frame holonomy H, a unit quaternion (Lemmas 7 and 12), together with its 64-bit SHD-CCP packet.
- **A3 — implied fourth.** The implied fourth of three engines is the closure $H_4 = (H_1H_2H_3)^{-1}$ of Lemma 7. Its geometric counterpart is the Klein-four coset closure of the FCC interleave, in which any three cosets determine the fourth.
- **A4 — phases.** The memory cycle has three 120° phase angles. Two are in use: onload and offload at the orthogonal handshake steps.

The hierarchy. It runs three data engines → one implied fourth, done three times:

```
Level 2 (optional):          G = (F_A F_B F_C)^-1     global fourth-of-
fourths (13th)
                                        |
Level 1:   F_A = (A1 A2 A3)^-1   F_B = (B1 B2 B3)^-1   F_C = (C1 C2 C3)^-1
                 |                       |                     |
Level 0:     A1  A2  A3            B1  B2  B3            C1  C2  C3
```

Quantity	Value
Data engines (level 0)	9
Implied fourths (level 1)	3
Total six-tier engines	12
Tier slots (12 × 6)	72
Outer frames (12 × WΣ)	720
Packet bits (12 × 64)	768
Data bits (9 × 64)	576

Each implied fourth is itself a full six-tier engine, as the request requires. It receives data by onload like any other engine. What differs is that its content is fixed by its three parents and offloads through the inverse tier system (§3).

2. Proposition M1 — the implied fourth is a parity engine

Statement. Let H_1, H_2, H_3 be the holonomies of three data engines and set $H_4 = (H_1H_2H_3)^{-1}$. Then $H_1H_2H_3H_4 = 1$, and each H_i is determined by the other three:

- $H_1 = (H_2H_3H_4)^{-1}$
- $H_2 = H_1^{-1} H_4^{-1} H_3^{-1}$
- $H_3 = H_2^{-1} H_1^{-1} H_4^{-1}$
- $H_4 = (H_1H_2H_3)^{-1}$

Proof. The product $H_1H_2H_3H_4 = 1$ is a relation in a group, so it is preserved by cyclic shifts: conjugating by H_1 gives $H_2H_3H_4H_1 = 1$, and so on for each rotation. Solving each cyclic form for its last factor gives the four formulas. ∎

Consequences.

1. **Single-erasure tolerance per group.** One lost engine in each of groups A, B and C is fully recoverable: up to three losses across the hierarchy.
2. **Two erasures in one group are fatal.** If H_1 and H_2 are both lost, every pair $(X, X^{-1}T)$ with $T = (H_3H_4)^{-1}$ is consistent with the record. The check found 50 distinct consistent pairs out of 50 trials.
3. **Capacity.** The fourth carries no new information, so only 9 of the 12 engines carry data (75%).
4. **Order matters.** Quaternion multiplication does not commute, so every engine in the hierarchy must use the same multiplication order. This is the open order convention of Lemma 12. The same convention must be fixed system-wide.
5. **Additive form.** The geometric version is the FCC coset closure: (1,1,0) + (1,0,1) + (0,1,1) = (2,2,2) ≡ (0,0,0) mod 2. This is parity in the plain XOR sense, and it is the same erasure code as the holonomy version.

Optional 13th engine. A global fourth-of-fourths $G = (F_A\ F_B\ F_C)^{-1}$ adds exactly one new recovery case: losing a data engine *and its own fourth* (for example A1 and F_A). G and F_B, F_C rebuild F_A, and then F_A rebuilds A1. It does not rescue two lost data engines in one group, because the group's product gives one equation in two unknowns.

Verified. 500 groups of exact rational unit quaternions: closure holds and all four recovery formulas are exact (no floating point).

3. Proposition M2 — offload onto the inverse tier system is exact

Statement. Onload carries data from the core out through tiers 0 → 4 by the transition word $W = U_3U_2U_1U_0$, where U_even = U and U_odd = U^{-1} (Lemma 1/7). The exact inverse tier system is $W^{-1} = U_0^{-1}U_1^{-1}U_2^{-1}U_3^{-1}$. Then:

1. **The inverse system is the same system.** Reversing the alternating word U, U^{-1}, U, U^{-1} and inverting each letter gives U, U^{-1}, U, U^{-1} again. So the inverse tier system has the same transitions as the original, traversed tier 4 → 0. No separate inverse hardware is needed.
2. **The net rotation is the identity.** W = I, and the spinor word is $h \cdot \bar{h} \cdot h \cdot \bar{h} = 1$ exactly, with h = (1−i+j+k)/2. Onload followed by inverse offload therefore leaves no residual rotation: no sign flip and no 2π ambiguity.
3. **The radial scale is exactly reversed.** $R_0 = 15$ and $R_4 = 155$, so onload scales by $R_4/R_0 = 31/3$ and inverse offload by 3/31. Both are exact rationals.
4. **Data is preserved at fixed width.** Left multiplication $q \mapsto hq$ is a norm-preserving bijection of the Hurwitz integers, and $\bar{h}(hq) = q$. A data quaternion passed through any number of tier rotations therefore stays a Hurwitz integer of the same norm and the same bit width. This is the contrast with Lemma 13: products of *data with data* grow in bit width, but rotation by unit spinors does not.

Proof. Items 1–3 are direct computation (checked exactly). For item 4, h is a Hurwitz unit (N(h) = 1). Multiplication by a unit maps the Hurwitz order onto itself, and N(hq) = N(h)N(q) = N(q). The inverse map is multiplication by $\bar{h} = h^{-1}$. ∎

What the implied fourth gives back. Offloading F_A returns exactly $F_A = (A1A2A3)^{-1}$: the product, not the three parents. The history-loss count from the system analysis applies: $24^3 = 13{,}824$ chains of Hurwitz units collapse to 24 products. To read an individual parent, three of the four engines in its group are needed (M1).

Verified. Transition-word identities and W = I with integer matrices; spinor word exactly 1; 5000 random Hurwitz data quaternions survive h, then $\bar{h}$, exactly.

4. Twelve cubes, 72 angles, 720 frames

Geometry of 12 = 3 × 4 (THEOREM). The FCC lattice built by the 3:1 interleave has 12 nearest neighbours, the cuboctahedron vectors (±1, ±1, 0) and their permutations. They split by which coordinate is zero into three planar groups of four. In each group the four vectors sum to zero, so any three determine the fourth: the additive form of "three engines plus their implied fourth". The 120° rotation about (1,1,1) carries each group onto another, which is the same symmetry that cycles the three phase angles (§5). The twelve-engine hierarchy is therefore the cuboctahedral shell of one engine in the tessellation, with each group of four lying in one coordinate plane.

Counts (CORRESPONDENCE).

Count	Value	Matches
12 engines × 6 tiers	72	72 middle-gear frames (720/10)
12 engines × WΣ = 60	720	720 outer frames
72 slots in 10 inner blocks	8, 7, 7, 7, 7, 8, 7, 7, 7, 7	the A_l block pattern
360° / 72	5° per slot	—

These numbers agree, but no lemma forces them, so they are tagged as correspondences.

Which "72" is meant. Two different 72s appear in the corpus:

- the **count** 72 = 12 × 6 tier slots (above);
- the **angle** 72°, the dihedral angle of the {4,3,5} hyperbolic cubic honeycomb (five cubes round each edge).

The twelve-engine hierarchy gives the first. The second is the stacking angle that makes the cube tessellation hyperbolic and unbounded. They agree only through 360°/5 = 72° versus 360°/72 = 5°, which is a coincidence of numerals rather than an identity. The monograph should say which is intended wherever "72 geometric angles" appears.

5. Proposition M3 — the unused 120° phase

Whether the third phase is "free" depends on what a phase angle physically is. There are two readings, and they give opposite answers.

(a) Phases as time or frame slots (free). If the three 120° phases are three disjoint slots in the cycle, each can carry a full packet independently. Using the third raises throughput by 3/2 at no cost to the other two.

(b) Phases as phasors on one complex carrier (not free). If data is written as amplitudes on e^0, $e^{2\pi i/3}$ and $e^{4\pi i/3}$ of one carrier, then $1 + \omega + \omega^2 = 0$. The carrier has only two real dimensions, so three phasors hold just two independent numbers, and the third phase is already spent.

Decomposition (THEOREM). Take three real channels (a, b, c), one per phase. Let $s = a + b + c$ and $z = a + \omega b + \omega^2 c$. Then:

- (a, b, c) is recovered exactly from s and z;
- a 120° cyclic shift of the channels leaves s fixed and multiplies z by ω.

So three phases = one invariant line ⊕ one rotating plane. Checked in 1000 random cases.

Match with the implied fourth. The FCC interleave has the same structure. Under the 120° rotation about (1,1,1), the corner coset o_0 is fixed and the three face cosets are cycled. The rotating plane carries the three data engines, which trade places under rotation. The invariant line carries what every rotation fixes, and that is the position of the implied fourth.

Recommended use of the third phase.

1. **Parity phase (preferred).** Onload and offload the implied fourth's traffic in the third phase. Both data phases then carry 100% data. Without this, parity takes a quarter of the two working phases, so data throughput is ×4/3 relative to sharing.
2. **Clock phase.** Alternatively, carry the reference clock (the caesium 30-pulse window of Lemma 13/Hoard 2026c) in the third phase, so timing never competes with data.
3. **Data phase.** Under reading (a) only, use it as a third data channel: ×3/2 throughput, but parity then has to share again.

Constraint from Lemma 8. The eight-step cycle has apex steps k = 2 and 6, which are 180° apart, not 120°. The 120° structure comes from the order-3 tier rotation, not from the eight-step cycle. The two align over a joint super-cycle of lcm(8, 3) = 24 steps. The phase plan must be written on that 24-step cycle: open item O2.

6. Memory budget, limits, open items

Budget per cycle.

Configuration	Data bits	Parity bits	Losses survived
12 engines, parity shares two phases	576	192	one per group
12 engines, parity in third phase	576 per two data phases (×4/3 data rate)	192 in phase 3	one per group
+ 13th global engine	576	256	one per group, or one data engine plus its own fourth
Third phase as data, reading (a) only	864	192	one per group

Limits that do not move.

- **Pigeonhole.** No arrangement stores more than the bits written. The hierarchy buys erasure tolerance and phase throughput, not extra capacity.
- **History loss.** A fourth stores a product; histories are not recoverable from it alone.
- **Two erasures in one group** are unrecoverable without an extra code.
- **Order convention.** Quaternion multiplication does not commute, so one multiplication order must be fixed for the whole hierarchy (Lemma 12).

Open items.

- **O1 — Physical meaning of a phase angle.** Slot (reading a) or phasor (reading b)? This decides whether the third phase is free.
- **O2 — 24-step super-cycle.** Write the onload, offload and third-phase schedule on lcm(8, 3) = 24 steps, so the 120° phases meet the k = 2, 6 apex steps of Lemma 8.
- **O3 — "5-manifold".** Does it add structure beyond the six tiers? Nothing here depends on it.
- **O4 — 13th engine.** Include the global fourth-of-fourths or not? The cost is one engine for one extra recovery case.
- **O5 — Which "72".** The count 72 or the honeycomb angle 72°?

Verification log (`verify_memory.py`, saved to the project):

```
[Parity] 500 exact rational groups: H1H2H3H4 = 1 and each of the four
recovered from the other three: True
[Parity] with H1 and H2 both lost, distinct (H1, H2) pairs consistent with
H3, H4: 50 (non-unique)
[Tiers] onload transitions U0..U3 = U, U^-1, U, U^-1: True ; product U3U2U1U0
= I: True ; inverse word identical to onload word: True
[Tiers] spinor word h, h^-1, h, h^-1 multiplies to exactly 1: True ; scale
product R4/R0 = 31/3 ; inverse scale 3/31
[Tiers] 5000 Hurwitz data quaternions: h*q stays Hurwitz, same norm, h-bar*
(h*q) = q exactly (no bit growth): True
[Cubocta] 12 vectors; groups by zero coordinate: [4, 4, 4] ; each group sums
to 0: True ; rotation about (1,1,1) maps groups to groups: True
[Counts] 12 x 6 = 72 ; W_Sigma = 60 ; 12 x W_Sigma = 720 ; 720/10 = 72 ;
inner blocks of the 72: [8, 7, 7, 7, 7, 8, 7, 7, 7, 7]
[Phase] 1 + w + w^2 = 0.0 (three phases on one complex carrier are dependent)
[Phase] 1000 cases: (a,b,c) recovered from sum + one complex number; a cyclic
shift fixes the sum and rotates z by 120 deg: True
[Phase] FCC cosets under the 120-degree rotation: corner coset fixed: True ;
face cosets cycled: True
[Budget] data fraction 9 of 12 = 3/4 ; packets 12 x 64 bits = 768 ; data bits
9 x 64 = 576 ; three phase slots vs two: x 3/2
```